Smart Hospital Portal - Architecture and Agent Flow

Purpose

This document explains the current architecture of the Smart Hospital Portal, when each agent is called, and when an agent is bypassed. It also highlights the profile-question routing issue shown in the UI screenshot, where the user repeatedly asked for name and age before the assistant answered correctly.

High-Level Architecture

1. The frontend in ‘app/api/static/app.js‘ collects the user’s chat message and current state.
2. The frontend sends a ‘POST /chat‘ request with the message, bearer token, and chat state.
3. ‘app/api/routes/chat.py‘ authenticates the user, attaches ‘patient_profile‘, loads chat history if needed, and loads active appointments.
4. ‘run_patient_chat()‘ in ‘app/agents/graph.py‘ starts the LangGraph workflow unless ‘chat_closed‘ is already true.
5. The ‘supervisor‘ node decides which specialist agent should handle the message.
6. The selected agent updates ‘GraphState‘ and usually returns to the supervisor.
7. If ‘final_response‘ is present or the supervisor routes to ‘finish‘, LangGraph ends the turn and the API returns the response and updated state.
8. The frontend renders the assistant response, quick action buttons, or the closed-chat modal.

Graph Structure

Entry point:

‘supervisor‘

Available graph nodes:

* ‘supervisor‘
* ‘triage_router‘
* ‘conversation_agent‘
* ‘remedy_agent‘
* ‘medical_rag‘
* ‘appointment_booker‘
* ‘finish‘

All specialist agents return control to ‘supervisor‘ after they update the state. The supervisor then either routes to another agent or ends the current turn.

Agent Responsibilities

Supervisor Agent

File: ‘app/agents/supervisor.py‘

Main responsibility:

* Decide whether the latest user message continues the current flow, interrupts it, or closes the chat.

Called:

* First on every chat turn.
* Again after every specialist agent returns.

Bypasses:

* Bypasses all agents if ‘final_response‘ already exists.
* Bypasses medical flow when the user confirms chat ending from ‘end_confirmation‘.
* Bypasses old flow when the user asks to end the chat by setting ‘awaiting=end_confirmation‘.
* Bypasses direct appointment booking to ‘medical_rag‘ if symptoms exist but department is unknown.

Important state fields:

* ‘awaiting‘
* ‘intent‘
* ‘symptoms‘
* ‘target_department‘
* ‘remedy_given‘
* ‘persisting‘
* ‘booking_active‘
* ‘chat_closed‘

Triage Router Agent

File: ‘app/agents/triage_router.py‘

Main responsibility:

* Extract intent, symptoms, and severity from the latest message.

Called when:

* No intent exists yet.
* User gives new symptoms.
* Supervisor routes a changed topic back to triage.

Bypasses:

* If the message is only a greeting, it replies directly and does not proceed to conversation/remedy.
* If intent is unclear, it asks whether the user is describing symptoms or wants booking.
* If symptoms and intent are extracted successfully, it does not produce a final response; it lets supervisor route next.

Current gap:

* Profile questions like "what is my name and age" can be classified as greeting or unclear because the schema only supports ‘greeting‘, ‘triage_symptoms‘, ‘direct_booking‘, and ‘unclear‘. There is no ‘profile_query‘ intent.

Conversation Agent

File: ‘app/agents/conversation_agent.py‘

Main responsibility:

* Ask follow-up intake questions until enough context exists for remedy or referral.

Called when:

* Symptoms exist but required context is incomplete.
* ‘awaiting=conversation‘.
* ‘intent‘ exists and ‘_conversation_complete()‘ is false.

Bypasses:

* If enough structured intake is already present, it returns without a final response so supervisor can route to remedy.
* If the patient asks for doctor/appointment during intake, it sets ‘intent=direct_booking‘ and exits.
* If max intake questions are reached, it treats context as sufficient.

Current issue shown in screenshot:

* The user asked for profile info, not medical intake. Because there is no account/profile route, the message entered triage/conversation behavior. The model eventually used ‘patient_profile‘, but only after repeated phrasing.

Recommended bypass:

* Add deterministic profile-query detection before triage:
* If user asks "what is my name", "my age", "my profile", "blood group", "health issues", route directly to a profile/account response.
* This should bypass ‘triage_router‘, ‘conversation_agent‘, ‘remedy_agent‘, ‘medical_rag‘, and ‘appointment_booker‘.

Remedy Agent

File: ‘app/agents/remedy_agent.py‘

Main responsibility:

* Generate personalized care tips and classify follow-up replies.

Called when:

* Intake is complete and ‘remedy_given‘ is false.
* ‘awaiting=remedy_check‘.
* User requests remedy while in booking menus.

Bypasses:

* If severity is ‘emergency‘, it bypasses normal remedies and tells user to seek emergency care.
* If follow-up says symptoms are improving, it closes that care path with a recovery message.
* If symptoms persist or worsen, it sets ‘persisting=True‘, allowing supervisor to route to ‘medical_rag‘ and then booking.
* If a confirmed booking exists, it should ask whether to forward new symptoms as a clinical note rather than offering another booking.

Medical RAG Agent

File: ‘app/agents/medical_rag.py‘

Main responsibility:

* Match symptoms and collected context to a hospital department.

Called when:

* User wants booking and symptoms exist but department is unknown.
* Symptoms are persisting after remedy and no department is set.

Bypasses:

* If RAG cannot confidently match a department, it routes back to ‘conversation_agent‘ by setting ‘awaiting=conversation‘.
* If a department is found, it only sets ‘target_department‘; supervisor then routes onward.

External dependency:

* Uses Qdrant-backed RAG through ‘app/services/rag.py‘.

Appointment Booker Agent

File: ‘app/agents/appointment_booker.py‘

Main responsibility:

* Handle symptom follow-up, doctor selection, slot selection, booking confirmation, and cancellation.

Called when:

* ‘intent=direct_booking‘ and department is known.
* ‘awaiting‘ is one of ‘symptom_follow_up‘, ‘doctor_selection‘, ‘slot_selection‘, or ‘cancellation_selection‘.
* User asks to cancel an appointment.
* Symptoms persist and a department is known.

Bypasses:

* If the patient asks for remedy while choosing doctor/slot, it clears booking state and routes to ‘remedy_agent‘.
* If the patient asks to cancel while in a booking menu, it jumps to cancellation flow.
* If no doctors are available, it returns a final response and does not ask for doctor selection.
* If no slots are available for a selected doctor, it stays in doctor selection.
* After booking or cancellation, it sets ‘awaiting=end_confirmation‘ instead of continuing the medical flow.

Common Flow Examples

New Symptom With Remedy

1. Frontend sends message.
2. ‘/chat‘ attaches user profile and history.
3. ‘supervisor‘ routes to ‘triage_router‘.
4. ‘triage_router‘ extracts symptoms and severity.
5. ‘supervisor‘ routes to ‘conversation_agent‘ if context is incomplete.
6. ‘conversation_agent‘ asks intake questions until enough information exists.

7. ‘supervisor‘ routes to ‘remedy_agent‘.
8. ‘remedy_agent‘ gives care tips and waits for improvement/persistence response.

Persisting Symptoms to Doctor Booking

1. User says symptoms are persisting.
2. ‘supervisor‘ routes to ‘remedy_agent‘ because ‘awaiting=remedy_check‘.
3. ‘remedy_agent‘ sets ‘persisting=True‘.
4. ‘supervisor‘ routes to ‘medical_rag‘ if no department exists.
5. ‘medical_rag‘ sets ‘target_department‘.
6. ‘supervisor‘ routes to ‘appointment_booker‘.
7. ‘appointment_booker‘ shows doctors, slots, and books the appointment.
8. After booking, ‘awaiting=end_confirmation‘.

Direct Booking

1. User asks for doctor or appointment.
2. ‘supervisor‘ routes to ‘triage_router‘ or ‘medical_rag‘ depending on available symptoms.
3. If department is unknown, ‘medical_rag‘ runs.
4. If department is known, ‘appointment_booker‘ runs.
5. Booking flow proceeds through doctor selection and slot selection.

Appointment Cancellation

1. User says cancel/delete/remove appointment.
2. ‘supervisor‘ routes to ‘appointment_booker‘.
3. ‘appointment_booker‘ lists active appointments.
4. User selects appointment number or reference.
5. ‘appointment_booker‘ cancels appointment and sets ‘awaiting=end_confirmation‘.

End Chat

1. User says "end the chat", "bye", "thank you", or similar.
2. ‘supervisor‘ asks for confirmation by setting ‘awaiting=end_confirmation‘.
3. User confirms with ‘yes‘, ‘sure‘, ‘ok‘, ‘confirm‘, etc.
4. ‘supervisor‘ sets ‘chat_closed=True‘.
5. Frontend disables the composer and shows the closed-chat modal.
6. User must click "Start new chat" to begin a clean chat.

Agent Bypass Scenarios

Scenario	Agent Called	Agents Bypassed	Why
Chat already closed	none after guard	all graph agents	‘run_patient_chat()‘ returns closed-chat message immediately
User confirms end chat	‘supervisor‘	triage, conversation, remedy, RAG, booking	‘awaiting=end_confirmation‘ and confirmation detected
User only greets	‘triage_router‘	conversation, remedy, RAG, booking	greeting response is final for that turn
User asks direct booking and department is known	‘appointment_booker‘	conversation, remedy, RAG	no need for intake or department matching
User asks direct booking and department is unknown	‘medical_rag‘	conversation and remedy	user wants doctor, not home remedy
RAG cannot match department	‘medical_rag‘ then ‘conversation_agent‘	appointment booking	more symptom detail is required
User asks remedy during booking menu	‘appointment_booker‘ then ‘remedy_agent‘	current booking selection	booking state is cleared and remedy

is prioritized |
| No doctors available | ‘appointment_booker‘ | doctor/slot selection |
no options exist to select |
| Emergency severity | ‘remedy_agent‘ | normal remedy generation |
emergency advice is returned immediately |
| Profile/account question | should be profile handler | all medical agents
| currently missing; should bypass medical routing |

Screenshot Issue: Name and Age Query

Observed behavior:

* User asked "what is my name and age" multiple times.
* Assistant repeatedly responded with generic hospital greeting or unclear triage question.
* Eventually it answered from ‘patient_profile‘.

Root cause:

* ‘patient_profile‘ is available in state, but the routing system has no profile/account intent.
* The triage schema does not include ‘profile_query‘.
* The supervisor prompt focuses on medical, remedy, booking, cancellation, and finish choices.
* As a result, a non-medical profile question may be treated as greeting or unclear.

Expected behavior:

* On the first message "what is my name and age", the system should answer directly:
* "Your name is Jaffer and you are 26 years old."

Recommended implementation:

* Add a deterministic check before triage in ‘supervisor_node‘ or ‘_dynamic_route‘.
* If the text asks about known profile fields, return a final response from ‘patient_profile‘.
* Suggested profile fields:
* ‘name‘
* ‘age‘
* ‘blood_group‘
* ‘health_issues‘
* ‘mobile_number‘
* ‘email‘
* ‘address‘

Recommended pseudo-flow:

‘‘‘text
if user asks profile/account question:
answer from state["patient_profile"]
return finish for this turn
else:
continue normal medical supervisor routing
‘‘‘

This profile response should not close the chat. It should end only the current turn.

State Fields That Control Routing

* ‘intent‘: high-level user goal such as symptoms or direct booking.

* ‘awaiting‘: the next expected user response type.
* ‘symptoms‘: extracted symptoms.
* ‘severity‘: urgency estimate.
* ‘target_department‘: department selected by RAG.
* ‘remedy_given‘: whether remedy advice has already been provided.
* ‘persisting‘: whether symptoms are persisting/worsening.
* ‘booking_active‘: whether a booking flow is in progress.
* ‘confirmed_booking‘: latest booked appointment.
* ‘confirmed_bookings‘: list of active confirmed bookings.
* ‘chat_closed‘: whether the entire chat is closed.
* ‘conversation_history‘: messages used for context.
* ‘patient_profile‘: logged-in user’s profile details.

Summary

The system is a LangGraph-based hospital assistant. The supervisor is the central router. Triage extracts symptoms, conversation gathers missing context, remedy gives care advice, RAG maps symptoms to departments, and appointment booker handles doctors, slots, booking, and cancellation. Agents are bypassed whenever state already provides enough information, the user changes intent, the chat is closed, or a special flow such as cancellation or end-chat takes priority. The main current routing gap is profile/account questions, which should bypass all medical agents and answer directly from ‘patient_profile‘ on the first attempt.